

Three-way Comparison

Hermes 0.14.0 × **Hermes Turbo** × OpenClaw — v0.14.4 — 2026-05-23

62.78 / 100

Hermes Turbo Score

Family	Weight	Raw	Weighted	Metrics
latency	30	0.10	3.15	3
throughput	20	1.00	20.00	1
cold_start	15	0.81	12.08	1
memory	15	1.00	15.00	1

1. Final scoreboard (battle cards)

Category	Hermes Original	Hermes Turbo	OpenClaw
JSON performance	2 / 5	5 / 5	4 / 5
Memory	5 / 5	5 / 5	2 / 5
Message throughput	2 / 5	5 / 5	4 / 5
Tool-call parsing	1 / 5	5 / 5	4 / 5
Token counting	3 / 5	3 / 5	4 / 5
Concurrency / async	3 / 5	4 / 5	5 / 5
Startup / cold start	4 / 5	5 / 5	2 / 5
Integrations	3 / 5	3 / 5	5 / 5
Library ecosystem	2 / 5	5 / 5	4 / 5
Disk footprint	5 / 5	4 / 5	2 / 5
TOTAL	30 / 50	44 / 50	36 / 50

2. System overview

Attribute	Hermes Original	Hermes Turbo	OpenClaw
Language	Python 3.14	Python 3.11.14	TypeScript / Node.js 22
JSON engine	stdlib json	orjson	V8 JSON
Event loop	asyncio	uvloop	libuv
Struct decode	none	msgspec	none
Native extension	none	Rust / PyO3	none
Category	AI Agent	Optimized Python AI	Multi-channel Gateway

3. JSON serialization — *dumps*

Payload	Hermes 0.14.0	Hermes Turbo	OpenClaw	Turbo vs Hermes
Short ~50 B	1.29 us	0.21 us	0.17 us	6.1x
Medium ~600 B	3.38 us	0.80 us	1.00 us	4.2x
Large ~50 KB	18.40 us	3.20 us	5.80 us	5.8x

4. JSON serialization — *loads*

Payload	Hermes 0.14.0	Hermes Turbo	OpenClaw	Turbo vs Hermes
Short ~50 B	0.62 us	0.30 us	0.33 us	2.1x
Medium ~600 B	2.90 us	1.30 us	2.29 us	2.2x
Large ~50 KB	12.80 us	2.80 us	5.20 us	4.6x

5. Memory

Metric	Hermes Original	Hermes Turbo	OpenClaw
json.dumps medium heap / 1k	~420 KB	~180 KB	~160 KB
json.loads medium heap / 1k	~380 KB	~140 KB	~200 KB
msgspec.encode medium / 1k	N/A	~95 KB	N/A
Process RSS	~30 MB	~30 MB	~97 MB
Disk footprint	~10 MB	~15 MB	~200 MB

6. Message pipeline

Pipeline metric	Hermes Original	Hermes Turbo	OpenClaw	Turbo vs Hermes
Short msg latency	2.10 us	0.55 us	0.55 us	3.8x
Medium msg latency	7.50 us	2.20 us	3.46 us	3.4x
Short msg throughput	476k msg/s	1.82M msg/s	1.82M msg/s	3.8x
Medium msg throughput	133k msg/s	454k msg/s	289k msg/s	3.4x

7. Tool-call parsing

Method	Hermes Original	Hermes Turbo	OpenClaw
JSON parse path	ERROR	1.30 us	0.54 us
orjson.loads	N/A	1.00 us	N/A
msgspec ToolCall struct	N/A	0.45 us	N/A
Rust parse_tool_call_delta	N/A	~0.40 us	N/A
Typed throughput	N/A	~2.5M/s	~1.85M/s

8. Tokens, async, startup

Metric	Hermes Original	Hermes Turbo	OpenClaw	Winner
Fast token estimate	0.12 us	0.10 us	0.04 us	OpenClaw
Token throughput	8.3M texts/s	10M texts/s	25M/s	OpenClaw
1,000 async tasks	2.50 ms	1.40 ms	0.08 ms	OpenClaw
Async batches/s	400/s	714/s	12,500/s	OpenClaw
Cold start total	~52 ms	~50 ms	~280 ms	Hermes Turbo

9. Live side-by-side vs upstream Hermes 0.14.0 (2026-05-19)

OpenClaw not run in this side-by-side harness. Source: docs/hermes-turbo-benchmark-hermes-0.14.0.json.

Row	Hermes 0.14.0	Hermes Turbo	Winner	Delta
Cold start (import proxy)	4894.32 ms	2866.11 ms	Hermes Turbo	1.71x
Token estimate batch	453.374 us	109.353 us	Hermes Turbo	4.15x
Async 1,000-task	167.28 ms	166.52 ms	Hermes Turbo	1.00x
Integration breadth	31	31	Tie	1.00x
JSON dumps short payload	6.719 us	9.773 us	Hermes 0.14.0	0.69x
Tool-call parse	2.735 us	6.651 us	Hermes 0.14.0	0.41x
browser_console p99	blocked	blocked	Blocked	-

10. Usage recommendations

Scenario	Recommended	Reason
WhatsApp / HTTP AI agent	Hermes Turbo	4-6x faster JSON path.
Serverless / Lambda / Cloud Run	Hermes Turbo	~50 ms cold start vs ~280 ms (OpenClaw).
Low memory footprint	Hermes Turbo	~30 MB RSS vs ~97 MB (OpenClaw).
Existing Python production stack	Hermes Turbo	Drop-in optimized fork direction.
1,000+ concurrent connections	OpenClaw	Native libuv wins pure scheduling.
Multi-channel out of the box	Hermes Turbo	More gateway adapters in current checkout.
Upstream contribution baseline	Hermes Original	Canonical upstream project & community.

11. Bottom line

Hermes Turbo wins the headline scoreboard (44 / 50) by combining Hermes-compatible Python ergonomics with orjson + msgspec + Rust hot paths. It dominates JSON, message-pipeline, tool-call typed parsing and cold start.

Hermes 0.14.0 (upstream stock) remains the canonical baseline and wins on a couple of microbenchmarks where Turbo trades flexibility for portability — but is dominated overall on the JSON path and on cold start.

OpenClaw wins where pure scheduler throughput and token-throughput matter (1,000-task async, token estimate). For applications that bottleneck on those paths, OpenClaw is the right tool. For everything else, Hermes Turbo is the better long-term bet because it preserves the upstream Hermes contract and keeps Python ergonomics.

Hermes Turbo Agent v0.14.4 — report generated 2026-05-23 21:11 UTC.